МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЗАХІДНОУКРАЇНСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
Факультет комп’ютерних інформаційних технологій
Кафедра комп’ютерних наук

ЗВІТ

про виконання лабораторної роботи №5
з дисципліни «Методи аналізу інтервальних даних»
на тему: «Програмна реалізація задачі синтезу допусків на основі методу допускового еліпсоїдного оцінювання»
Варіант 1

Виконав:
студент групи ІПЗМПм-11
Боднар А.

Перевірила:
к.т.н., доцент
Крепич С. Я.

Тернопіль – 2026

МЕТА РОБОТИ

Вивчити метод допускового еліпсоїдного оцінювання та його застосування до задачі синтезу допусків на параметри статичної системи; навчитися обчислювати матрицю чутливості, вибирати насичений блок і будувати еліпсоїдну оцінку допускової області; реалізувати метод у вигляді програмної системи з графічним інтерфейсом.

ЗАВДАННЯ

Розробити програмну систему для реалізації задачі синтезу допусків на основі методу допускового еліпсоїдного оцінювання. У програмній системі передбачити:

– зручний графічний інтерфейс;
– перевірку коректності введених даних;
– виведення знайдених допусків на параметри.

ВИХІДНІ ДАНІ

Таблиця 1 – Дані варіанта 1

Параметр	Значення
Вихідна характеристика	y = a·(x − 1)^3 + b·x^2 − c·x + 4
Номінальні значення параметрів	a = 1, b = 0,5, c = −2
Коридор відхилень	ε = 10 %
Точки x	3; 4; 5; 6; 7; 8

Точки x у завданні не задані; обрано рівномірну сітку на проміжку, де характеристика визначена й не наближається до нуля.

ТЕОРЕТИЧНІ ВІДОМОСТІ

Розв’язок задачі синтезу допусків на параметри системи зводиться до пошуку області взаємозв’язаних допустимих відхилень параметрів, яка належить допусковій області Ω. Цю область апроксимують еліпсоїдом: перевагою

еліпсоїдної оцінки є гладкість функціональних коридорів і економне використання пам’яті — достатньо зберігати матрицю конфігурації еліпсоїда та його центр.

Вимоги до точності характеристик задаються коридором відхилень від номінальних значень $\delta y_i^- \le S_i \cdot \delta b \le \delta y_i^+$, $i = 1 \dots N$, де S — матриця чутливості з елементами

$$S_{ij} = b_j \cdot \partial y / \partial b_j \,(x_i),$$

а δb — вектор відносних відхилень параметрів. Допускову еліпсоїдну оцінку подають m-вимірним еліпсоїдом

$$Q_m = \{\, \delta b : (\delta b - \delta b_0)^{T} \cdot H \cdot (\delta b - \delta b_0) \le r \,\}.$$

Матрицю конфігурації H будують на основі насиченого блоку S_m — m рядків матриці чутливості, для яких об’єм гіперпаралелепіпеда Ω_m мінімальний:

$$\Pi (\delta y_i^+ - \delta y_i^-)^2 \cdot \det(S_m \cdot S_m^{T})^{-1} \to \min,$$

$$E = \operatorname{diag}(0{,}5 \cdot (\delta y_i^+ - \delta y_i^-)), \quad H = S_m^{T} \cdot E^{-2} \cdot S_m.$$

Оскільки коридор симетричний відносно номінальних значень, центр еліпсоїда — у нулі, а його радіус

$$r = \min (\Delta_i)^2 / (S_i^{T} \cdot H^{-1} \cdot S_i), \quad \Delta_i = \min\{-\delta y_i^-;\ \delta y_i^+\},$$

забезпечує дотик вписаного еліпсоїда до граней допускової області. Допуски на параметри:

$$\delta b_j = r / \sqrt{h_{jj}} \cdot 100\ \%,$$

де h_{jj} — діагональні елементи матриці H.

ХІД РОБОТИ

1. Коридор відхилень і матриця чутливості

Відхилення $\delta y_i\pm = \pm 10\ \% \cdot |y_i|$; чутливості обчислено за формулою $S_{ij} = b_j \cdot \partial y / \partial b_j$ (таблиця 2).

Таблиця 2 – Коридор відхилень і матриця чутливості

x	y	δy^-	δy^+	Sa	Sb	Sc
3	22,5	−2,25	2,25	8	4,5	6
4	47	−4,7	4,7	27	8	8

x	y	δy^-	δy^+	Sa	Sb	Sc
5	90,5	−9,05	9,05	64	12,5	10
6	159	−15,9	15,9	125	18	12
7	258,5	−25,85	25,85	216	24,5	14
8	395	−39,5	39,5	343	32	16

2. Вибір насиченого блоку

Перебрано всі 20 блоки з 3 рядків; у таблиці 3 наведено п’ять з найменшим критерієм.

Таблиця 3 – Критерій для блоків матриці чутливості

Точки x	$\Pi(\delta y^+ - \delta y^-)^2 \cdot \det(S_m \cdot S_m^T)^{-1}$
3; 4; 8	3,0933
3; 5; 8	3,2486
3; 4; 7	4,8211
3; 5; 7	6,4041
3; 6; 8	6,9441

Насичений блок утворюють точки x = 3; 4; 8:

	Sa	Sb	Sc
x = 3	8	4,5	6
x = 4	27	8	8
x = 8	343	32	16

3. Матриця конфігурації еліпсоїда

$E^{-2} = \mathrm{diag}(0{,}197531;\ 0{,}045269;\ 0{,}000641)$, $H = S_m^T \cdot E^{-2} \cdot S_m$:

	a	b	c
a	121,0473	23,9241	22,777
b	23,9241	7,5535	8,5587
c	22,777	8,5587	10,1724

4. Радіус еліпсоїда і допуски

Відношення $\Delta_i^2/(S_i^T \cdot H^{-1} \cdot S_i)$ для точок x = 3; 4; 5; 6; 7; 8: 1; 1; 0,9191; 1,2384; 1,3178; 1. Мінімум досягається в точці x = 5:

$$r = 0{,}91911.$$

$$\delta a = r/\sqrt{haa} \cdot 100\ \% = 0{,}91911/\sqrt{121{,}0473} \cdot 100\ \% = 8{,}35\ \%;$$

$$\delta b = r/\sqrt{hbb} \cdot 100\ \% = 0{,}91911/\sqrt{7{,}5535} \cdot 100\ \% = 33{,}44\ \%;$$

$$\delta c = r/\sqrt{hcc} \cdot 100\ \% = 0{,}91911/\sqrt{10{,}1724} \cdot 100\ \% = 28{,}82\ \%;$$

Таблиця 4 – Знайдені допуски на параметри

Параметр	Номінал	Допуск, %	Допустимий інтервал
a	1	8,35 %	[0,9165; 1,0835]
b	0,5	33,44 %	[0,3328; 0,6672]
c	−2	28,82 %	[−2,5763; −1,4237]

5. Перевірка на прикладі з лекції

Програму перевірено на прикладі з лекції 7 ($y = a(x - 2)^2 + bx + 2$, $a = -0{,}3$, $b = 3$, коридор 15 %, $x = 3; 5; 7; 9; 11$). Коридор, матриця чутливості, матриця H і результат $r = 0{,}31$, $\delta a = 1{,}89$ %, $\delta b = 1{,}3$ % збігаються з лекцією, якщо взяти блок зі слайда ($x = 9; 11$). Водночас для цього блоку на слайді вказано критерій −0,6072, хоча він не може бути від’ємним (добуток квадратів, поділений на $\det(S_m \cdot S_m^T) > 0$); правильне значення — 0,00649. Найменший критерій має блок $x = 3; 11$, і для нього $r = 1$, $\delta a = 6{,}6$ %, $\delta b = 4{,}69$ %. У програмі насичений блок обирається автоматично, а будь-який інший блок можна вибрати вручну.

6. Програмна реалізація

Програмну систему реалізовано як вебзастосунок мовою TypeScript з бібліотеками React і React-Bootstrap; вона працює в браузері й розміщена на GitHub Pages. Обчислювальна частина не залежить від інтерфейсу:

– func.ts — розбір формули характеристики, введеної текстом, зі змінною x і параметрами a, b, c (створений у лабораторній роботі №3 і доповнений параметрами);

– lab5.ts — дані 20 варіантів, перевірка вхідних даних, обчислення визначника й оберненої матриці методом Гаусса, перебір блоків і функція solveLab5, що повертає коридор, матрицю чутливості, насичений блок, матрицю H, радіус і допуски.

Інтерфейс має три кроки: введення моделі, параметрів, коридору й точок; коридор, матриця чутливості, таблиця критеріїв усіх блоків і матриця H; знайдені допуски з допустимими інтервалами параметрів і графіком коридору. Програма перевіряє формулу, числові поля, кількість і повторення точок, область визначення

характеристики, нульові параметри та попереджає про точки з дуже вузьким коридором; якщо параметри входять у модель залежно (наприклад, b·x і c·x у варіантах 9 і 15), усі блоки вироджені й програма пояснює, що окремі допуски не визначаються. Правильність перевірено автоматичними тестами (Vitest), зокрема на прикладі з лекції.

Сторінка роботи на сайті: https://4ndruxa.github.io/university-labs/#/maid/lab5.

Репозиторій з кодом: https://github.com/4ndruxa/university-labs.

7. Результати роботи програми

На рисунку 1 наведено вікно введення вихідних даних.

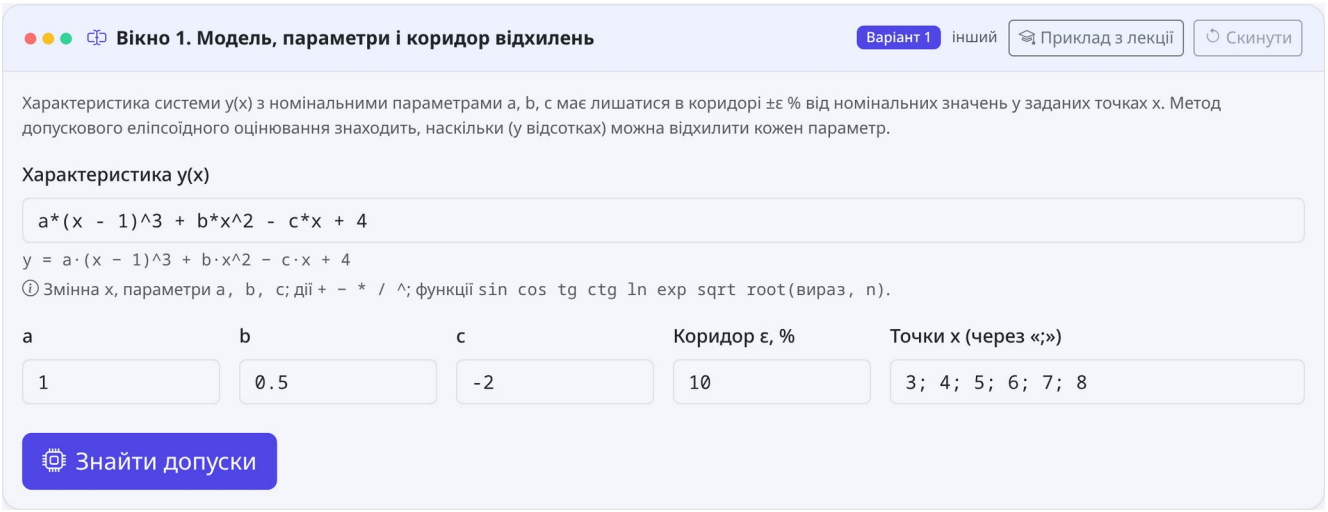


Рисунок 1 – Вікно введення моделі, параметрів і коридору

На рисунку 2 наведено коридор відхилень, матрицю чутливості, вибір насиченого блоку та матрицю H.

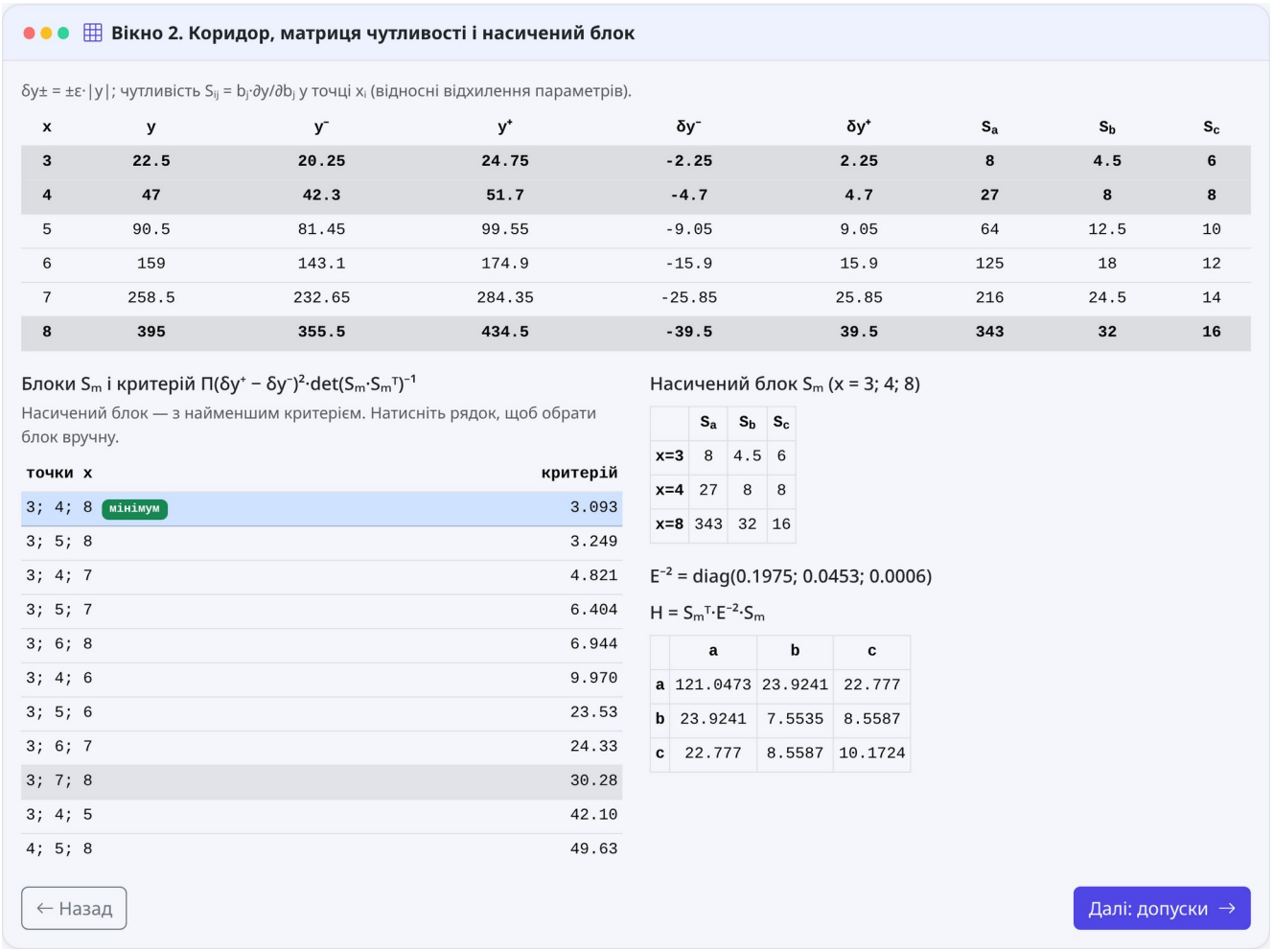


Рисунок 2 – Вікно з матрицею чутливості і насиченим блоком

На рисунку 3 наведено знайдені допуски на параметри.

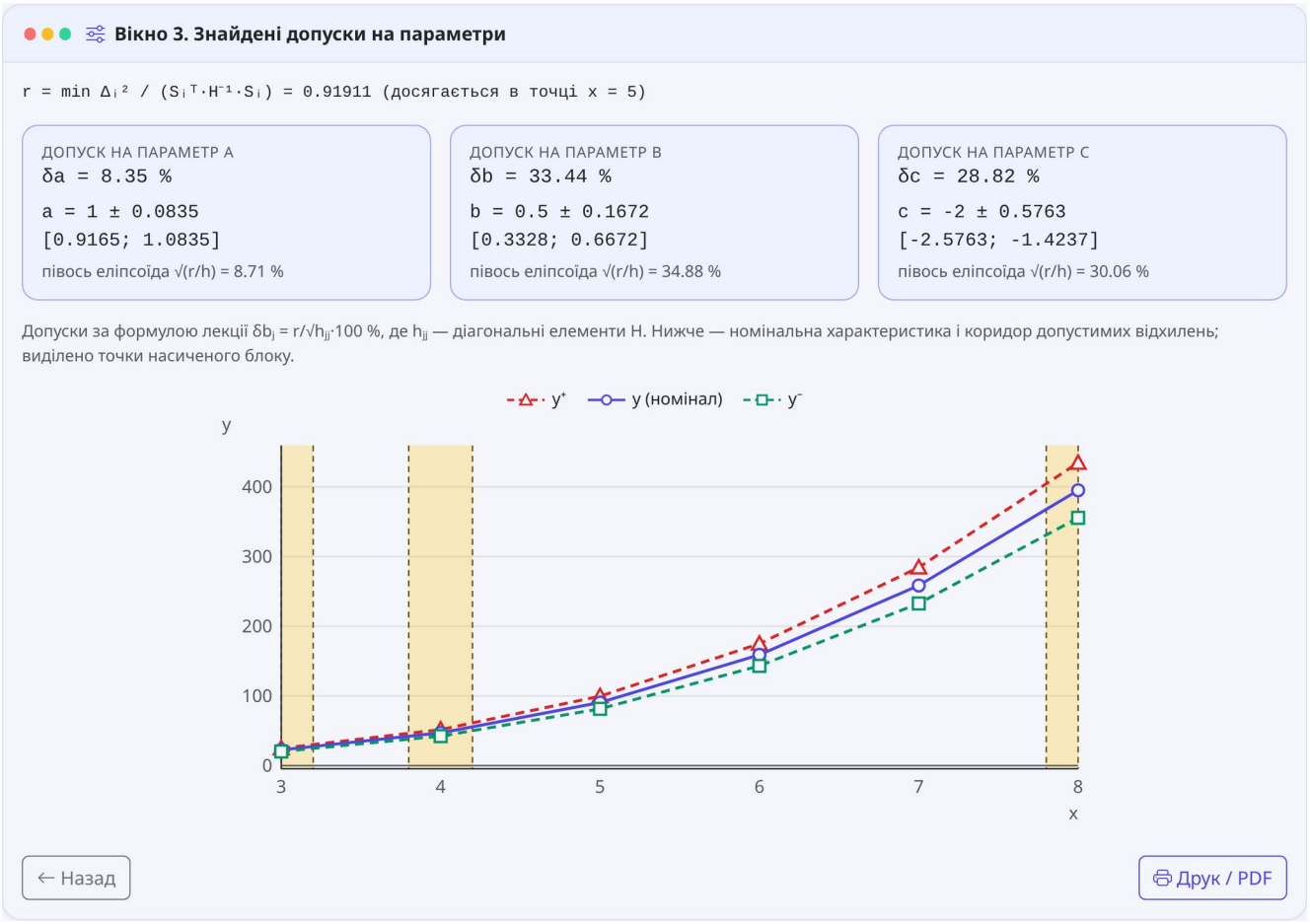


Рисунок 3 – Вікно зі знайденими допусками на параметри

ВИСНОВКИ

У ході лабораторної роботи вивчено метод допускового еліпсоїдного оцінювання і розроблено програмну систему, яка за моделлю характеристики, номінальними параметрами і коридором відхилень знаходить допуски на параметри.

Для варіанта 1 отримано допуски δa = 8,35 %, δb = 33,44 %, δc = 28,82 %. Найжорсткіший допуск має параметр a, до якого характеристика найчутливіша; радіус еліпсоїда обмежує точка x = 5.

Перевірка на прикладі з лекції показала, що програма відтворює лекційні проміжні результати, а вибір насиченого блоку за правильно обчисленим критерієм дає ширші допуски, ніж на слайді.

ДОДАТОК А. ЛІСТИНГ ПРОГРАМНОГО КОДУ

Нижче наведено обчислювальну частину програми з коментарями. Модуль розбору формул func.ts наведено у звіті до лабораторної роботи №3; повний код застосунку розміщено в репозиторії https://github.com/4ndruxa/university-labs.

Файл lab5.ts

```
// ЛР5: синтез допусків на параметри методом допускового еліпсоїдного оцінювання
    (лекція 7)
//
//   δyᵢ± = ±ε·|y(xᵢ)|                          — коридор відхилень характеристики
    від номінальної
//   Sᵢⱼ = bⱼ·∂y/∂bⱼ (xᵢ)                       — матриця чутливості (відносні
    відхилення параметрів)
//   Sₘ: m рядків з N, Π(δyᵢ⁺ − δyᵢ⁻)²·det(Sₘ·Sₘᵀ)⁻¹ → min   — насичений блок
//   E = diag(0,5·(δyᵢ⁺ − δyᵢ⁻)), i ∈ блок;   H = Sₘᵀ·E⁻²·Sₘ   — матриця
    конфігурації еліпсоїда
//   Δᵢ = min{−δyᵢ⁻; δyᵢ⁺};   r = min Δᵢ² / (Sᵢᵀ·H⁻¹·Sᵢ)     — радіус еліпсоїда
//   δbⱼ = r / √hⱼⱼ · 100 %                      — допуски на параметри
import { DomainError, evalAt, parseFunction, type Node } from './func'

export const LAB5_PARAMS = ['a', 'b', 'c'] as const
/** Точки x, у яких задається коридор (у завданні не вказані; для варіанта 5 — 2…
    7, бо y(8) ≈ 0) */
export const LAB5_X = [3, 4, 5, 6, 7, 8]

export interface Lab5Variant {
  y: string
  a: number
  b: number
  c: number
  eps: number // коридор відхилень, %
  x?: number[]
}

export const LAB5_VARIANTS: readonly Lab5Variant[] = [
  { y: 'a*(x - 1)^3 + b*x^2 - c*x + 4', a: 1, b: 0.5, c: -2, eps: 10 },
  { y: 'a*sin(x) + b*x^2 - c*x^3 - 3', a: 0.3, b: -3, c: 1.5, eps: 10 },
  { y: 'a*x + b*ln(x) - c*(x - 1) + 0.5', a: 0.1, b: -1, c: 2, eps: 10 },
  { y: 'b*(x - 1)^3 + c*x^2 - a*x', a: 2.5, b: 0.5, c: -0.7, eps: 10 },
  { y: 'c*(x - 5)^3 + a*x - b*sqrt(x) + 0.6', a: -0.2, b: 0.5, c: 0.1, eps: 15, x:
    [2, 3, 4, 5, 6, 7] },
  { y: 'a/(x - 2) + b*cos(x^2) - c*x + 2', a: 0.5, b: -2, c: 1.5, eps: 15 },
  { y: 'a*(x - 1)^3 + b/x^2 - c*(x - 3)', a: 1.5, b: 3, c: -0.2, eps: 15 },
  { y: 'a*(x - 1.5)^2 + b*x^3 - c/x + 1', a: -0.4, b: 1, c: 0.5, eps: 5 },
  { y: 'a*sin(3x) - b*x - c*x + 3', a: 0.1, b: -0.4, c: -2, eps: 5 },
  { y: 'a*(x - 1)^3 - b*x^2 + c*x + 4', a: -2, b: 1.5, c: -0.4, eps: 5 },
  { y: 'a*cos(2x) + b*(x - 0.5)^2 - c*(x + 1)', a: -0.2, b: 2.5, c: 2, eps: 10 },
  { y: '-a*(x - 1)^3 + b*(x - 6)^2 - c*ln(x) - 1', a: -0.4, b: -1, c: -2, eps:
    10 },
  { y: 'c*(x - 1)^3 + b*cos(x^2) - a*x + 4', a: 2, b: 2.5, c: -0.7, eps: 10 },
  { y: 'a*x^4 + b*x^2 - c*(x - 1) + 3', a: 0.5, b: -1, c: 1.5, eps: 5 },
  { y: 'a*x + b*sin(x) - c*x', a: -0.7, b: 0.5, c: 0.1, eps: 5 },
  { y: 'a*tg(x) + b*x^2 + c*x + 3', a: -0.4, b: 3, c: 0.5, eps: 5 },
  { y: 'a*sqrt(x - 1) + b*x^2 + c/x + 7', a: 1, b: 0.5, c: -1, eps: 15 },
  { y: 'a*x^3 + b*(x - 1)^2 + c*x - 2', a: 0.5, b: -0.2, c: 3, eps: 15 },
  { y: 'a*sin(2x) + b*x^2 - c/x + 0.5', a: -0.4, b: 0.5, c: 2, eps: 15 },
```

```
  { y: 'a*(x - 2)^3 + b*(x - 1) - c*x^2 + 3', a: 0.1, b: -2, c: -0.7, eps: 5 },
]

// ---------- Невелика лінійна алгебра (матриці m×m, m ≤ 3–4) ----------

export type RealMatrix = number[][]

export const transpose = (A: RealMatrix): RealMatrix => A[0].map((_, j) =>
    A.map((row) => row[j]))
export const mul = (A: RealMatrix, B: RealMatrix): RealMatrix => A.map((row) =>
    B[0].map((_, j) => row.reduce((s, v, k) => s + v * B[k][j], 0)))

/** Визначник методом Гаусса з вибором головного елемента */
export function det(A: RealMatrix): number {
  const M = A.map((row) => [...row])
  const n = M.length
  let d = 1
  for (let k = 0; k < n; k++) {
    let p = k
    for (let i = k + 1; i < n; i++) if (Math.abs(M[i][k]) > Math.abs(M[p][k])) p =
    i
    if (M[p][k] === 0) return 0
    if (p !== k) {
      ;[M[p], M[k]] = [M[k], M[p]]
      d = -d
    }
    d *= M[k][k]
    for (let i = k + 1; i < n; i++) {
      const f = M[i][k] / M[k][k]
      for (let j = k; j < n; j++) M[i][j] -= f * M[k][j]
    }
  }
  return d
}

/** Обернена матриця методом Гаусса — Жордана */
export function inverse(A: RealMatrix): RealMatrix {
  const n = A.length
  const M = A.map((row, i) => [...row, ...Array.from({ length: n }, (_, j) => (i
    === j ? 1 : 0))])
  for (let k = 0; k < n; k++) {
    let p = k
    for (let i = k + 1; i < n; i++) if (Math.abs(M[i][k]) > Math.abs(M[p][k])) p =
    i
    if (Math.abs(M[p][k]) < 1e-14) throw new Error('Матриця вироджена — оберненої
    не існує')
    ;[M[p], M[k]] = [M[k], M[p]]
    const piv = M[k][k]
    for (let j = 0; j < 2 * n; j++) M[k][j] /= piv
    for (let i = 0; i < n; i++) {
      if (i === k) continue
      const f = M[i][k]
      for (let j = 0; j < 2 * n; j++) M[i][j] -= f * M[k][j]
    }
  }
  return M.map((row) => row.slice(n))
}

/** Усі сполучення з n по m (індекси) */
export function combinations(n: number, m: number): number[][] {
  const out: number[][] = []
```

```
  const pick = (start: number, acc: number[]) => {
    if (acc.length === m) { out.push(acc); return }
    for (let i = start; i < n; i++) pick(i + 1, [...acc, i])
  }
  pick(0, [])
  return out
}

// ---------- Метод ----------

export interface Lab5Input {
  y: Node
  params: readonly string[] // назви параметрів, напр. ['a', 'b', 'c']
  values: Record<string, number> // номінальні значення
  eps: number // %
  x: number[]
  /** Примусовий насичений блок (індекси точок) — щоб відтворити вибір зі слайда
    лекції */
  block?: number[]
}

export interface Lab5Block {
  rows: number[] // індекси точок
  criterion: number | null // null — блок вироджений
}

export interface Lab5Result {
  y: number[]
  dyMinus: number[]
  dyPlus: number[]
  S: RealMatrix // N×m
  blocks: Lab5Block[]
  block: number[] // обраний насичений блок
  bestBlock: number[] // блок з найменшим критерієм
  Sm: RealMatrix
  E2: number[] // діагональ E⁻²
  H: RealMatrix
  Hinv: RealMatrix
  ratios: number[] // Δᵢ² / (Sᵢᵀ·H⁻¹·Sᵢ) для кожної точки
  r: number
  rIndex: number // точка, на якій досягається мінімум
  tolerances: number[] // δbⱼ = r/√hⱼⱼ, %
  semiAxes: number[] // √(r/hⱼⱼ), % — перетин еліпсоїда з віссю bⱼ
}

/** Матриця чутливості: bⱼ·∂y/∂bⱼ (центральна різниця; для лінійних за параметрами
    моделей — точно) */
function sensitivity(inp: Lab5Input): RealMatrix {
  return inp.x.map((x) =>
    inp.params.map((name) => {
      const v = inp.values[name]
      const h = 1e-6 * Math.max(1, Math.abs(v))
      const up = evalAt(inp.y, x, { ...inp.values, [name]: v + h })
      const down = evalAt(inp.y, x, { ...inp.values, [name]: v - h })
      return (v * (up - down)) / (2 * h)
    }),
  )
}

/** Помилки (блокують розрахунок) і попередження */
```

```
export function validateLab5(inp: Lab5Input): { errors: string[]; warnings:
    string[] } {
  const errors: string[] = []
  const warnings: string[] = []
  const m = inp.params.length
  if (!(inp.eps > 0 && inp.eps < 100)) errors.push('Коридор ε — число від 0 до 100
    %')
  for (const name of inp.params) {
    const v = inp.values[name]
    if (!Number.isFinite(v)) errors.push(`Параметр ${name} — число`)
    else if (v === 0) errors.push(`Параметр ${name} = 0: відносний допуск для
    нульового параметра не визначений`)
  }
  if (inp.x.some((x) => !Number.isFinite(x))) errors.push('Точки x — числа')
  else if (new Set(inp.x).size !== inp.x.length) errors.push('Точки x не повинні
    повторюватися')
  if (inp.x.length < m) errors.push(`Потрібно щонайменше ${m} точки x (за
    кількістю параметрів)`)
  if (errors.length) return { errors, warnings }
  const ys: number[] = []
  for (const x of inp.x) {
    try {
      ys.push(evalAt(inp.y, x, inp.values))
    } catch (e) {
      errors.push(`y(${x}) не обчислюється: ${(e as Error).message}`)
    }
  }
  if (errors.length) return { errors, warnings }
  const maxY = Math.max(...ys.map(Math.abs))
  if (maxY === 0) errors.push('y = 0 в усіх точках — коридор вироджений')
  ys.forEach((v, i) => {
    if (v === 0) errors.push(`y(${inp.x[i]}) = 0 — коридор у цій точці нульовий`)
    else if (Math.abs(v) < 0.05 * maxY) warnings.push(`y(${inp.x[i]}) ≈ $
    {v.toPrecision(3)} — дуже вузький коридор у цій точці обмежить усі допуски`)
  })
  return { errors, warnings }
}

export function solveLab5(inp: Lab5Input): Lab5Result {
  const { errors } = validateLab5(inp)
  if (errors.length) throw new Error(errors.join('\n'))
  const m = inp.params.length
  const y = inp.x.map((x) => evalAt(inp.y, x, inp.values))
  const dyPlus = y.map((v) => (inp.eps / 100) * Math.abs(v))
  const dyMinus = dyPlus.map((v) => -v)
  const S = sensitivity(inp)
  const width = (i: number) => dyPlus[i] - dyMinus[i]

  // критерій для кожного блоку; блок вироджений, якщо det(Sₘ·Sₘᵀ) ≈ 0 відносно
    масштабу стовпців
  // (параметри можуть мати чутливості різних порядків, тому масштаб — окремо для
    кожного стовпця)
  const colMax = inp.params.map((_, j) => Math.max(...S.map((row) =>
    Math.abs(row[j]))))
  const scale = colMax.reduce((p, v) => p * v * v, 1)
  const blocks: Lab5Block[] = combinations(inp.x.length, m).map((rows) => {
    const Sm = rows.map((i) => S[i])
    const d = det(mul(Sm, transpose(Sm)))
    if (!(Math.abs(d) > 1e-12 * scale)) return { rows, criterion: null }
    return { rows, criterion: rows.reduce((p, i) => p * width(i) ** 2, 1) / d }
  })
  const valid = blocks.filter((b) => b.criterion !== null)
  if (!valid.length) {
```

```
    throw new DomainError('Усі блоки матриці чутливості вироджені: параметри
    входять у модель залежно (наприклад, b·x і c·x) — окремі допуски не
    визначаються')
  }
  const bestBlock = valid.reduce((best, b) => (b.criterion! < best.criterion! ?
    b : best)).rows
  const block = inp.block ?? bestBlock
  if (block.length !== m || block.some((i) => !(i >= 0 && i < inp.x.length)))
    throw new Error(`Блок має містити ${m} різні точки`)

  const Sm = block.map((i) => S[i])
  const E2 = block.map((i) => 1 / (0.5 * width(i)) ** 2)
  const H = mul(transpose(Sm), Sm.map((row, k) => row.map((v) => v * E2[k])))
  const Hinv = inverse(H)
  const ratios = S.map((Si, i) => {
    const delta = Math.min(-dyMinus[i], dyPlus[i])
    const q = Si.reduce((s, v, p) => s + v * Si.reduce((t, w, q2) => t + Hinv[p]
    [q2] * w, 0), 0)
    return delta ** 2 / q
  })
  const rIndex = ratios.reduce((best, v, i) => (v < ratios[best] ? i : best), 0)
  const r = ratios[rIndex]
  return {
    y,
    dyMinus,
    dyPlus,
    S,
    blocks,
    block,
    bestBlock,
    Sm,
    E2,
    H,
    Hinv,
    ratios,
    r,
    rIndex,
    tolerances: H.map((row, j) => (r / Math.sqrt(row[j])) * 100),
    semiAxes: H.map((row, j) => Math.sqrt(r / row[j]) * 100),
  }
}

/** Вхідні дані з текстової формули */
export function lab5FromText(y: string, values: Record<string, number>, eps:
    number, x: number[], params: readonly string[] = LAB5_PARAMS): Lab5Input {
  return { y: parseFunction(y, params), params, values, eps, x }
}

/** Які з параметрів трапляються у формулі (у порядку списку) */
export function usedParams(node: Node, params: readonly string[] = LAB5_PARAMS):
    string[] {
  const seen = new Set<string>()
  const walk = (n: Node): void => {
    if (n.k === 'param') seen.add(n.name)
    else if (n.k === 'neg') walk(n.a)
    else if (n.k === 'bin' || n.k === 'pow') { walk(n.a); walk(n.b) }
    else if (n.k === 'fn') n.args.forEach(walk)
  }
  walk(node)
  return params.filter((p) => seen.has(p))
}
```

```
/** Приклад з лекції 7 — для перевірки на захисті */
export const LAB5_LECTURE: Lab5Variant = { y: 'a*(x - 2)^2 + b*x + 2', a: -0.3, b:
    3, c: 1, eps: 15, x: [3, 5, 7, 9, 11] }
```